

Qt 프로젝트 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.14

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 프로젝트에서는, 코드 구조 설계, C++Qt&ROS2 코드 작성, 오류 수정 및 UI 구성 보조 도구로 사용하였다. 처음에는 A*알고리즘을 이용하여 로봇 시뮬레이터를 구현하려 하였으나 비슷한 구현을 C 언어로 한 경험이 있었기에 게임의 형태로 기능을 더 추가하여 구현하기로 결정하였다. 이후 프로그램의 방향과 개발 우선순위를 정한 뒤 코드 구성 및 작성 부분 등에서 AI의 도움을 받았다.

UI는 가능한 한 직접 작성하려 하였다. Qt 창에서 기본적인 ui 구성 틀을 잡고 버튼과 텍스트 등을 배치하였다. 이후 디자인적인 요소(색깔 및 테두리 마감)는 AI의 도움을 받아 UI를 완성하였다. UI를 완성한 후 ROS2 패키지 전환을 AI에게 맡겼다.

2. AI 사용 부분

2.1 구조 설계

과제의 C++, Qt, Ros2 사용 조건과 노드 간 통신 조건을 제시하고 정한 주제와 UI를 AI에게 전달하였다. AI는 GUI 노드와 게임 엔진 노드를 나누고, 이동 명령과 게임 상태를 토픽으로, 재시작을 서비스로 주고받는 구조를 제안하였다. 미로는 매번 달라지되, 탈출 가능한 길이 있어야 한다는 요구를 AI에게 전달하였고, AI는 격자형 미로 생성과 경로 확인 방법을 설명한 뒤 구현에 반영하였다.

Qt의 **Style Sheet(QSS)**를 사용했습니다. 완전한 검정이 아니라 눈이 덜 피로한 진한 청흑색 #0b1115 입니다.

Qt Designer에서 **MainWindow**를 선택하고:

1. Property Editor에서 **styleSheet** 검색
2. 값 오른쪽의 **...** 버튼 클릭
3. 다음 내용 입력

```
</> CSS
MainWindow,
QWidget#centralWidget {
    background-color: #0b1115;
    color: #e8f0f3;
}
```

게임 화면과 패널도 각각 다른 어두운 색을 적용했습니다.

```
</> CSS
QGraphicsView#gameView {
    background-color: #0c1418;
}

QFrame#headerFrame,
QFrame#gameFrame,
QFrame#sidePanel {
    background-color: #10181d;
}
```

현재 설정은 **mainwindow.ui (line 22)**의 **styleSheet** 속성에 들어 있습니다. Qt 스타일시트는 CSS와 비슷하며 **#gameView** 처럼 **objectName**을 이용해 특정 위젯만 꾸밀 수 있습니다.

📄 ↻ ⏪ (금요일) 오후 9:34

2.2 ROS2 패키지 구현

UI 구성을 완료하고 어느정도 AI 에게 코딩을 맡겨 코드를 구현하고 완성시킬즈음 Qt 전용 프로젝트에서 하고있었다는 점을 깨달았고 이후 패키지 이전을 요청했다. AI 는 최종 robot-maze_game 패키지에 game_gui_node 와 game_engine_node, player_commandgame_state 토픽, restart_game 서비스를 구성하였다. 이 단계는 내가 ROS2 조건을 확인하고 이전 시점을 결정했으며, 파일 작성과 빌드 과정은 AI 가 주로 수행하였다.

2.3 실제 피드백과 수정

프로그램을 실행한 뒤 보이는 문제를 구체적으로 전달하며 수정하였다.

방향키나 wasd 를 누를때 기존에는 키보드 자체키를 인식하여 로봇이 움직이는 방법을 사용했었다. 그러자 키를 꼭 누르면 한번움직인후 잠시 멈췄다가 다시 쪽이동하는 문제가 발생하였다. 따라서 이를 키 입력 직후 150ms 간격으로 계속 입력되도록 방식을 조정하였다.

간헐적으로 로봇이 빠르게 움직이는 경우도 있었다. 방향 전환시 즉시 이동과 타이머가 겹치는 문제였었고 이또한 AI 에게 전달하여 해결하였다.

odt파일 확인해봐
그리고 궁금한점 있음
장애물은 어떻게 구성해야할까? 미로노점으로 구성하고 싶긴 한데 랜덤으로 항상 탈
출할수 있는길이 1~3개정도 있는 랜덤한 미로를 구성할 수 있나?

Worked for 1m 48s >

ODT를 확인했습니다. ROS2 QT GUI C++ 패키지, 노트 작성. odt에는 ROS 2 Qt GUI C++ 패키지 작성 가이드 링크가 들어 있습니다.

앞으로 패키지를 만들 때 해당 가이드에 맞춰 다음 구조를 따르겠습니다.

- ros2_create_qt_pkg 로 패키지 생성
- Qt GUI의 ROS 처리는 QNode 객체가 담당
- ROS spin은 QThread 안에서 실행
- MainWindow → QNode 접근은 멤버 함수 사용
- QNode → MainWindow 전달은 Qt Signal/Slot 사용
- 별도의 GameEngineNode 를 추가하여 노트 2개 조건 충족

장애물은 격자형 타일 맵으로 만드는 것이 가장 좋습니다. 각 칸을 다음 값으로 관리하면 됩니다.

C++

```
enum class TileType {  
    Wall,  
    Floor,  
    Start,  
    Exit,  
    Item  
};
```

예를 들어 21 x 21 크기의 2차원 vector 를 사용하고, Qt에서는 QPainter 로 각 칸을 사각형으로 그림니다.

- 검은색 또는 진한색: 벽
- 밝은색: 이동 가능한 길
- 파란색: 플레이어
- 빨간색: 적
- 노란색: 아이템
- 초록색: 출구

↓
랜덤 미로도 충분히 만들 수 있습니다. 가장 적합한 방법은 Randomized DFS 미로 생성 알고리즘입니다.

3. 깨달은 점

이번 프로젝트를 통해 Qt 화면과 ROS 2 게임 엔진이 역할을 나누고, 토픽과 서비스로 정보를 주고받는 구조를 이해했다. 화면의 입력이 게임 규칙에 반영되고, 바뀐 상태가 다시 화면에 표시되기까지의 흐름을 배웠다.

미로는 무작위로 만드는 것뿐 아니라 출구까지 갈 수 있는 길을 보장해야 한다는 점을 알게 되었다. 적의 이동을 살펴보면 BFS 같은 탐색 알고리즘이 게임 규칙에 실제로 어떻게 쓰이는지도 배웠다.

